Engineering Manager Interview Guide

Situational and behavioral scenarios with model answers, follow-up probes, and evaluation notes

How to use this guide

Each scenario contains a **Situation**, the **Question** an interviewer would ask, a **Model Response** (written in first person as a strong candidate would answer, loosely following the STAR format: Situation, Task, Action, Result), **Follow-up Probes** to test depth, and **What I Listen For / Red Flags** from the interviewer's perspective. Candidates should adapt the model answers with their own real experiences. Interviewers should use the same scenarios consistently across candidates for fair comparison.

#	Scenario	Core competency
1	Managing an Underperforming Senior Engineer	People management, accountability, empathy
2	Leading Through a Major Production Incident	Crisis leadership, communication, operational excellence
3	Resolving Conflict Between Two Engineers	Conflict resolution, fairness, team health
4	Unrealistic Deadline from Product or Leadership	Stakeholder management, estimation, negotiation
5	Technical Debt vs. Feature Delivery	Prioritization, technical judgment, business alignment
6	Promotion Expectations of a High Performer	Career development, honest feedback, retention
7	Influencing Without Authority Across Teams	Cross-functional leadership, dependency management
8	Low Morale After a Reorganization or Layoffs	Leadership under uncertainty, trust, retention

1. Managing an Underperforming Senior Engineer

Situation
You inherit a team. One senior engineer, well-liked and with deep system knowledge, has missed three consecutive sprint commitments, and their code reviews are slow and dismissive. The team has started routing around them.

Interview question: Walk me through how you would handle this, and tell me about a time you actually did.

Model response

Situation. At my last company I took over a team of eight. A staff-level engineer, the only person who understood our billing pipeline, had slipped on deliverables for about two months and junior engineers had stopped asking for his reviews.

First, diagnose before judging. I held a private 1:1 and opened with observations, not conclusions: 'I have noticed three sprints with slipped work and some friction in reviews. I want to understand what is going on from your side.' I learned he had been pulled into constant ad-hoc support requests that were invisible on the board, and he was burned out and quietly resentful that nobody had noticed.

Second, set clear expectations and support. I wrote down what good looked like: reviews turned around within one business day, and a defined share of his time for planned work. I took on the support burden by creating a rotating on-call/support role and routing requests through a queue. I also coached him on review tone, with specific examples of comments that landed as dismissive.

Third, create a checkpoint, not a threat. We agreed on a 6-week check-in with concrete signals. I documented the conversations and kept my skip-level and HR informed so that, if things did not improve, a formal plan would not be a surprise to anyone.

Result. Within six weeks his delivery was back on track, review turnaround dropped from about four days to one, and he began mentoring two juniors on the billing system. Had it not improved, my next step would have been a formal performance improvement plan with clear criteria and dates.

Lesson. Most performance problems are either clarity problems, capacity problems or motivation problems. Treating the wrong one wastes everyone's time, so I diagnose first.

Follow-up probes

- What if he had denied there was any problem?
- How did you make sure the team did not see this as favoritism or as a punishment?
- At what point do you move from coaching to a formal PIP?

What I listen for
Diagnosis before action, direct and kind communication, written expectations, involvement of HR at the right time, concern for the rest of the team.

Red flags
Jumping straight to termination, blaming the individual without examining the system, avoiding the conversation, no concrete outcome.

2. Leading Through a Major Production Incident

Situation
It is 2 a.m. and your checkout service is down. Revenue is being lost every minute. The on-call engineer has paged you, two other engineers are debugging in different directions, and executives are asking for updates in Slack.

Interview question: What do you do in the first 30 minutes, and what do you do afterwards?

Model response

Establish command. The most common failure in an incident is too many people doing too many things with no coordination. I would declare an incident, name an incident commander (me, or the most senior on-call person if I am not hands-on), and assign roles: someone who investigates, someone who communicates, and someone who scribes a timeline.

Stabilize before diagnosing. The first question is 'what changed?' If a deploy or config change went out recently, rollback is almost always faster than a forward fix. I would ask the team to prioritize mitigation (rollback, feature flag off, failover, scale up) over finding root cause.

Protect the engineers from noise. I would put stakeholder communication in a single channel and post updates at a fixed cadence, for example every 15 minutes, in the form of: current impact, what we know, what we are doing, next update time. That way executives do not have to ask and engineers are not interrupted.

After resolution. Within 48 hours we hold a blameless postmortem. We build a timeline, identify contributing factors (not 'the person who deployed'), and agree on a few high-leverage actions with owners and due dates, such as better alerts, canary deploys, or a runbook. I also check on the on-call engineer's wellbeing and give them time to recover.

Real example. A bad database migration once took down a core service for 40 minutes. We rolled back within 25 minutes of the page. The postmortem led to mandatory migration reviews, a staging replay of production traffic, and an alert on error-rate thresholds that cut our detection time from 12 minutes to under 2.

Follow-up probes

- What if a rollback is not possible?
- An executive is demanding you put your best engineer on it personally. What do you do?
- How do you keep postmortem action items from being ignored?

What I listen for
Structure and calm, mitigation over root cause, clear communication, blameless culture, follow-through on action items.

Red flags
Hero mentality, blaming individuals, no communication plan, skipping the postmortem.

3. Resolving Conflict Between Two Engineers

Situation
Two strong engineers disagree about an architecture decision (event-driven vs synchronous APIs). The discussion has turned personal, and each is lobbying other team members. The decision is blocking a project.

Interview question: How do you handle it?

Model response

Separate the people from the problem. I would first meet each engineer individually to understand their reasoning and what they are worried about. Often the stated disagreement is technical, while the underlying concern is something else, such as ownership, being heard, or fear of being on call for a design they do not trust.

Reframe around shared goals and criteria. I would bring them together and agree on what we are optimizing for: latency, operational simplicity, team familiarity, and time to market. Once criteria are explicit, the debate becomes about evidence, not about who is right.

Timebox and make it testable. If the data is unclear, I would ask for a short spike or a written design doc from each side, with a review date. A two-day proof of concept is far cheaper than weeks of argument.

Decide and commit. If they cannot converge, it is my job as the manager to make the call, or to involve the tech lead or architect, and explain the reasoning openly. I would use 'disagree and commit', document the decision in an ADR (architecture decision record), and note the conditions under which we would revisit it.

Address the behavior separately. The lobbying and personal tone is a behavior issue. I would give each of them private, specific feedback and reset team norms for technical debate.

Outcome. In a similar case, the prototype showed that the synchronous approach met our latency target with much lower operational cost. The engineer whose idea was not chosen led the follow-up work on observability, which kept him invested.

Follow-up probes

- What if one of them is clearly your strongest performer?
- What if you personally disagree with both approaches?
- How do you stop it recurring?

What I listen for
Curiosity about root causes, a decision-making process, willingness to decide, separate handling of behavior vs. technical issues.

Red flags
Taking sides based on seniority, splitting the difference to avoid conflict, never deciding.

4. Unrealistic Deadline from Product or Leadership

Situation
Your VP of Product commits a major feature to a customer for a date that your team estimates is about twice as long as available. The date is already public.

Interview question: What do you do?

Model response

Understand the commitment. I would ask what exactly was promised, to whom, and why that date. Often the date is firmer in someone's head than in the contract, and the scope of the promise is vaguer than people think.

Make the estimate credible. I would sit with the team, break the work down, and identify the critical path and risks. I would present the VP with a range and confidence level, not a single number, along with the assumptions behind it.

Bring options, not just a no. The levers are scope, time, resources and quality, and I refuse to trade quality silently. I would offer three options: (a) a reduced-scope version by the original date, such as the core flow for the customer's primary use case; (b) the full scope on a later date; (c) the full scope on the date with added staff, being honest that adding people to a late project has diminishing returns and that onboarding will cost us the first weeks.

Communicate early and in writing. The worst outcome is a surprise in the last week. I would agree on milestones that give early signals, and share weekly progress against them.

Hold the line on sustainability. If leadership still chooses the aggressive path, I would make sure it is a conscious, documented risk acceptance, and I would protect the team by limiting any crunch to a defined window with time off afterwards.

Result. In a similar case we shipped the core flow by the public date, and the remainder three weeks later. The customer cared about their top use case, which we had confirmed by calling them directly.

Follow-up probes

- What if the VP says 'just make it work'?
- How do you handle it if your team has a history of underestimating?
- When is crunch ever acceptable?

What I listen for
Options-based negotiation, data-driven estimates, early escalation, protecting the team without being obstructive.

Red flags
Silent acceptance followed by burnout, flat refusal, hiding bad news until late.

5. Technical Debt vs. Feature Delivery

Situation
Your team's main service is increasingly fragile. Deploys take three hours, incidents are frequent, and onboarding takes months. Product wants new features every sprint and sees refactoring as 'no business value'.

Interview question: How do you balance tech debt against roadmap pressure?

Model response

Translate debt into business terms. 'The code is messy' will lose every time. I would quantify the cost: hours lost per week to slow deploys, incident count and revenue impact, time to onboard a new engineer, and the delay on recent features caused by the legacy code. For example, 'we lose roughly 15 percent of engineering capacity to this, and three of the last five features slipped because of it'.

Prioritize the debt that is on the path of the roadmap. Not all debt is equal. I would rank items by pain times frequency and by how directly they block upcoming features. Refactoring code nobody touches is low priority.

Use a mix of mechanisms. (1) A standing capacity allocation, for example 15 to 20 percent of each sprint, agreed with product up front. (2) Refactor-as-you-go: when a feature touches a fragile area, the estimate includes making it safer. (3) Occasional dedicated projects for large items, framed with a clear business outcome such as 'cut deploy time from three hours to 20 minutes'.

Show progress. I would track a few metrics, such as deploy frequency, lead time, change failure rate, and incident count, and report them alongside feature delivery. When product sees lead time fall, support for the investment follows.

Result. In my last role we ran a 'deploy pipeline' project over two quarters. Deploys dropped from three hours to 25 minutes, and feature throughput increased about 30 percent the following quarter, which turned product into an advocate.

Follow-up probes

- What if product refuses any capacity allocation?
- How do you decide between rewrite and incremental refactor?
- How do you prevent new debt?

What I listen for
Quantification, pragmatism, partnership with product, awareness of rewrite risk.

Red flags
Treating all debt as equally urgent, no metrics, 'engineers know best' attitude.

6. Promotion Expectations of a High Performer

Situation
One of your best engineers asks to be promoted to senior. Their output is excellent, but they rarely mentor, do not influence design decisions beyond their own tasks, and have been difficult to work with in cross-team settings. They hint they may leave if they are not promoted.

Interview question: How do you respond?

Model response

Take the request seriously and respond with clarity. I would meet promptly and thank them for raising it. Then I would review the level expectations for senior engineer together, line by line, and assess honestly where they exceed, meet, or fall short of each.

Be specific and kind. I would say: 'Your technical execution is at senior level. The gaps are scope of influence and collaboration. For example, in the March design review you made a strong technical case but did not bring along the platform team, and the project stalled.' Specific examples are what make the feedback acceptable and actionable.

Create a concrete development plan. I would define two or three opportunities to demonstrate the missing skills: lead a cross-team design, mentor a new hire, and run the next design review. I would agree on timing, say six months, and what evidence would be reviewed, and I would give regular feedback along the way.

Do not promise what I cannot deliver. Promotions go through calibration, and I will not give a guarantee. I would commit to advocating for them if they show the evidence.

On the threat of leaving. I would not make decisions under pressure, since promoting someone who is not ready can hurt them (they struggle at the next level) and hurt team fairness. I would acknowledge the retention risk, explore other motivators such as scope and compensation within the level, and accept that they may leave if they value the title more than the path.

Result. A person in this situation stayed, led a cross-team migration, and was promoted in the next cycle with strong calibration support.

Follow-up probes

- What if the company has no budget for promotions this cycle?
- What if they leave anyway?
- How do you stay consistent across your reports?

What I listen for
Honest feedback with evidence, development plans, integrity under retention pressure, fairness.

Red flags
Promising a promotion to retain, vague feedback, ignoring the behavior gaps.

7. Influencing Without Authority Across Teams

Situation
Your project depends on an API change from the Platform team, which has a different roadmap and is unwilling to prioritize your request. Your launch is in eight weeks.

Interview question: How do you get what you need?

Model response

Understand their world. I would meet the Platform team's lead to learn their priorities, constraints, and what success looks like for them. Often a request is deprioritized because the benefit to them is unclear.

Make the case with shared value. I would frame the change in terms of company outcomes and what Platform gains, for example that three other teams have similar needs and a clean API would reduce their support burden. I would bring data, not opinions.

Reduce their cost. I would offer to do the work myself: have one of my engineers implement the change in their codebase following their standards, with their review. This often turns an unfunded request into a manageable code review.

Agree on the commitment explicitly. We would put in writing what will be delivered, by when, and who the contact is, and I would schedule short check-ins.

Have a fallback. I would design an interim solution, such as an adapter in our service, that we can ship if Platform slips, and flag the risk early to my leadership, so that if we need escalation it is a joint priority discussion between directors, not a complaint.

Result. In practice, we contributed the change ourselves, Platform reviewed it in a week, and we launched on time. The relationship improved, and they later sent us a similar contribution.

Follow-up probes

- What if they flatly refuse?
- When do you escalate?
- How do you build these relationships before you need them?

What I listen for
Empathy for other teams, creative problem solving, escalation as a last resort, contingency planning.

Red flags
Going straight to escalation, blaming the other team, no plan B.

8. Low Morale After a Reorganization or Layoffs

Situation
Following a company-wide layoff, your team lost two members. The remaining engineers are anxious, workload has increased, and your best engineers are quietly interviewing elsewhere.

Interview question: What are your first steps over the next month?

Model response

Acknowledge reality. I would hold a team meeting and say plainly that this was hard, that I do not have every answer, and what I can and cannot share. Pretending everything is fine destroys trust faster than uncomfortable honesty.

One-on-one conversations with everyone. Within the first week, I would speak to each person: how are you doing, what worries you, what do you need? I would listen far more than I speak, and follow up on what I hear.

Reduce the load. I would work with product and leadership to cut scope and stop low-value work. Doing the same with fewer people is not a plan. I would be explicit about what we are no longer doing and communicate that to stakeholders.

Provide as much clarity as possible. I would share the team's priorities, the company's financial reality to the extent I am allowed, and what I know about job security. I would never promise that there will be no further layoffs.

Retain and recognize. I would advocate for retention for critical people, with compensation, scope, or development opportunities, and give visible recognition for the extra effort. I would also model sustainable behavior by protecting time off.

Rebuild momentum. Within a month, I would pick a small, meaningful win the team can ship, to restore a sense of progress and confidence.

Result. After a similar event, I kept 90 percent of the team through the following year, and our engagement survey score for 'I trust my leadership' recovered within two quarters.

Follow-up probes

- What would you do if you had been told to keep information confidential that the team wanted?
- How do you handle your own stress?
- What if a top performer resigns?

What I listen for
Honesty, empathy, practical load reduction, authenticity.

Red flags
Spin, toxic positivity, expecting heroics from the remaining team.

Evaluation Scorecard

Rate each dimension from 1 (does not meet) to 4 (exceeds). Score independently before debriefing with the panel.

Dimension	What strong looks like	Score
People leadership	Diagnoses before acting, gives specific feedback, develops people, handles hard conversations	1 2 3 4
Execution and delivery	Realistic planning, clear prioritization, manages risk and scope openly	1 2 3 4
Technical judgment	Understands trade-offs, asks for evidence, balances debt and delivery	1 2 3 4
Communication	Clear, honest, adapts to audience, communicates bad news early	1 2 3 4
Influence and collaboration	Builds relationships across teams, escalates appropriately, offers options	1 2 3 4
Resilience and integrity	Stays calm under pressure, owns mistakes, resists shortcuts that harm people	1 2 3 4

General tips for candidates

- Use real examples with measurable results. Specific numbers make an answer credible.
- Spend about 20 percent of your answer on the situation and 80 percent on what you did and what happened.
- Own your mistakes and say what you learned. Managers who never fail are not believed.
- Say 'I' for your own actions and 'we' for team results, and be clear about the difference.
- Prepare questions for the interviewer about team health, expectations, and how success is measured.